[SOLUTION TEMPLATE] Assignment 2: Policy Gradients

提交截止日期见课程站点

提交入口：`https://oc.sjtu.edu.cn/courses/95065`

最终仅提交由本文件编译得到的 PDF 报告；无需提交代码、数据、实验日志或 TeX 源文件。

1 Policy Gradients (CartPole Environment)

In this section, you will evaluate the basic design choices in policy gradients: trajectory-centric returns, reward-to-go, advantage normalization, and batch size. To keep the workload manageable, we use four representative configurations.

```
# 1. Vanilla policy gradient: trajectory-centric return, no normalization
python sjtuRL2026/scripts/run_hw2.py --env_name CartPole-v0 -n 100 -b 1000 \
    --exp_name cartpole_vanilla

# 2. Reward-to-go
python sjtuRL2026/scripts/run_hw2.py --env_name CartPole-v0 -n 100 -b 1000 -rtg \
    --exp_name cartpole_rtg

# 3. Reward-to-go + advantage normalization
python sjtuRL2026/scripts/run_hw2.py --env_name CartPole-v0 -n 100 -b 1000 -rtg -na \
    --exp_name cartpole_rtg_na

# 4. Reward-to-go + advantage normalization + larger batch size
python sjtuRL2026/scripts/run_hw2.py --env_name CartPole-v0 -n 100 -b 4000 -rtg -na \
    --exp_name cartpole_rtg_na_lb
```

Where:

`-n`: number of iterations

`-b`: batch size

`-rtg`: use reward-to-go

`-na`: use advantage normalization

`exp_name`: name of the experiment

- Create one graph comparing the learning curves (average return vs. number of environment steps) for the four experiments above.

  For all plots in this assignment, the x-axis should be number of environment steps, logged as `Train_EnvstepsSoFar` (*not* number of policy gradient iterations).

- Answer the following questions briefly:
  - Does reward-to-go improve performance compared with vanilla trajectory-centric returns?
  - Does advantage normalization help?
  - Does increasing the batch size from 1000 to 4000 improve stability or final performance?
- Provide the exact command line configurations you used to run your experiments, including any parameters changed from their defaults.
- The best configuration should converge to a maximum score close to 200.

2 Neural Network Baseline (HalfCheetah)

You will now implement a value function as a state-dependent neural network baseline. This neural network will be trained in the update method of MLPPolicyPG along with the policy gradient update. We will train the baseline network for multiple gradient steps for each policy update, determined by the parameter `baseline_gradient_steps`.

No baseline

```
python sjtuRL2026/scripts/run_hw2.py --env_name HalfCheetah-v4 \
    -n 100 -b 5000 -rtg --discount 0.95 -lr 0.01 \
    --exp_name cheetah
```

Baseline

```
python sjtuRL2026/scripts/run_hw2.py --env_name HalfCheetah-v4 \
    -n 100 -b 5000 -rtg --discount 0.95 -lr 0.01 \
    --use_baseline -blr 0.01 -bgs 5 --exp_name cheetah_baseline
```

You might notice that we omitted-na (normalize advantages). That' s because in reality, advantage normalization is a very powerful trick, and eliminates the need for a baseline most of the simple environments that we test in.

- Plot a learning curve for the baseline loss.
- Plot a learning curve for the eval return. You should expect to achieve an average return over 300 for the baselined version.
- (bonus) Run another experiment with a decreased number of baseline gradient steps (`-bgs`) and/or baseline learning rate (`-blr`). How does this affect (a) the baseline learning curve and (b) the performance of the policy?
- **Optional:** Add `-na` back to see how much it improves things. Also, set `video_log_freq 10`, then open TensorBoard and go to the "Images" tab to see some videos of your HalfCheetah walking along!

3 Generalized Advantage Estimation (LunarLander-v2) (bonus)

You will now use the value function you previously implemented to implement a simplified version of GAE-λ. This will require filling in the remaining TODO section in `pg_agent.py:estimate_advantage`.

You will now use your implementation of policy gradient with generalized advantage estimation to learn a controller for a version of LunarLander-v2 with noisy actions. Search over $\lambda \in [0, 0.95, 0.98, 0.99, 1]$ to replace <todo> below. Do not change any of the other hyperparameters (e.g. batch size, learning rate).

```
python sjtuRL2026/scripts/run_hw2.py \
    --env_name LunarLander-v2 --ep_len 1000 \
    --discount 0.99 -n 300 -l 3 -s 128 -b 2000 -lr 0.001 \
    --use_reward_to_go --use_baseline --gae_lambda <todo> \
    --exp_name lunar_lander_lambda<todo>
```

- Provide a single plot with the learning curves for the `LunarLander-v2` experiments that you tried. Describe in words how λ affected task performance. The run with the best performance should achieve an average score close to 200 (180+).
- Consider the parameter λ. What does $\lambda = 0$ correspond to? What about $\lambda = 1$? Relate this to the task performance in `LunarLander-v2` in one or two sentences.

4 Hyperparameter Tuning (InvertedPendulum) (bonus)

First, train a policy for the inverted pendulum problem without GAE and with otherwise default settings. Use five different seeds to get a good estimate of average performance

```
for ($seed = 1; $seed -le 5; $seed++) {
  python sjtuRL2026/scripts/run_hw2.py --env_name InvertedPendulum-v4 -n 100 \
    --exp_name "pendulum_default_s$seed" \
    -rtg --use_baseline -na \
    --batch_size 5000 \
    --seed $seed
}
```

Your task is to tune hyperparameters so that your implementation reaches maximum performance (score of 1000) in fewer environment steps than these default settings (note: this is not the same as minimizing number of policy gradient iterations: one policy gradient iteration corresponds to batch size environment steps). Try and find hyperparameters that reach a score of 1000 in as few steps as possible !

1. Provide a set of hyperparameters that achieve high return on `InvertedPendulum-v4` in as few environment steps as possible.
2. Show learning curves for the average returns with your hyperparameters and with the default settings, with environment steps on the x-axis. Returns should be averaged over 5 seeds.

5 提交说明：仅提交 PDF 报告

请把实验图表、命令配置和文字答案填写到 `hw2_template.tex` 中，并使用任意能够正确编译本模板的 LaTeX 环境生成 PDF。最终文件应为 `hw2_template.pdf`。

课程平台只需上传填写完整且能够正常打开的 PDF 报告。无需上传 Python 代码、实验数据、日志、模型文件、TeX 源文件或 ZIP 压缩包。代码和实验输出仅用于完成实验与生成报告内容，请自行保留。

提交前请逐页检查 PDF，确认图表、公式、命令和答案均显示完整。提交入口：`https://oc.sjtu.edu.cn/courses/95065`。截止日期以课程平台通知为准。